실전으로 배우는 빅데이터와 AI

5

웹 서비스와 GPT 연동

© PMI. All rights reserved.
무단 복제 및 배포를 금합니다.

웹 서비스와 GPT 연동

- **웹 서비스와 인공지능**

지금까지 우리는 Python, 데이터 처리, 그리고 기본적인 프로그램을 통해 데이터를 분석하고 결과를 출력하는 방법을 학습했습니다. 하지만 실제 환경에서는 단순한 프로그램을 넘어, 사용자가 직접 사용할 수 있는 "웹 서비스 형태"로 구현하는 것이 중요합니다.

웹 서비스는 인터넷을 통해 누구나 접근할 수 있으며, 사용자의 입력을 받아 서버에서 처리한 뒤 결과를 다시 전달하는 구조로 동작합니다.

- **웹 서비스와 인공지능(GPT)의 활용**

최근에는 인공지능 기술의 발전으로, 웹 서비스에 GPT와 같은 AI 모델을 결합하여 사용자와 대화하고 문제를 해결하는 서비스가 증가하고 있습니다. 기존의 웹 서비스가 정해진 기능을 수행하는 데 집중했다면, GPT를 활용한 웹 서비스는 사용자의 질문을 이해하고 상황에 맞는 답변을 생성하는 지능형 서비스로 발전하고 있습니다.

GPT는 사용자의 자연어 입력을 이해하고 그 의미를 분석하여 적절한 답변을 생성하는 인공지능 모델입니다. 사용자가 문장 형태로 질문을 입력하면, GPT는 질문의 의도를 파악하고 관련 정보를 바탕으로 자연스럽게 답변을 제공합니다. 이러한 특징 덕분에 다양한 형태의 질문에도 유연하게 대응할 수 있습니다.

웹 서비스에서 GPT는 사용자의 질문을 해석하고 답변을 생성하는 역할을 수행하며, 이전 대화 내용을 바탕으로 맥락을 유지하는 대화형 기능도 제공합니다. 이를 통해 단순한 정보 제공을 넘어 사용자와 상호작용하는 서비스 구현이 가능합니다.

이러한 GPT 기반 웹 서비스는 교육, 의료, 고객 상담 등 다양한 분야에서 활용되고 있으며, 사용자는 복잡한 기능을 탐색하지 않고도 질문을 입력하는 것만으로 필요한 정보를 얻을 수 있습니다.

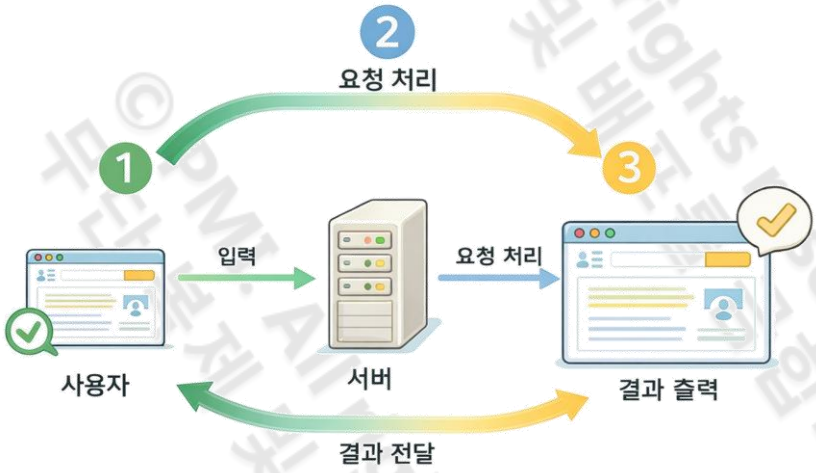
- **웹 서비스 동작 구조**

웹 서비스는 사용자의 입력을 받아 서버에서 처리한 뒤 결과를 다시 전달하는 클라이언트-서버 구조로 동작합니다.

먼저 사용자가 웹 브라우저에서 버튼을 클릭하거나 정보를 입력하면, 해당 데이터는

웹 서비스와 GPT 연동

JavaScript를 통해 서버로 전송됩니다. 서버는 전달받은 요청을 분석하고 필요한 작업을 수행하며, 경우에 따라 데이터베이스 조회나 외부 API 호출을 함께 진행합니다.



이후 서버는 처리 결과를 다시 클라이언트로 반환하고, 웹페이지는 이를 화면에 출력하여 사용자에게 보여줍니다. 이러한 과정은 매우 빠르게 반복되며, 사용자는 실시간으로 결과를 확인할 수 있습니다.

이와 같은 구조를 통해 웹 서비스는 별도의 프로그램 설치 없이도 다양한 기능을 제공할 수 있으며, 사용자와 서버 간의 지속적인 상호작용이 가능해집니다.

이와 같은 웹 서비스의 동작 구조를 이해하면, 사용자의 입력이 어떻게 서버로 전달되고 처리되는지, 그리고 결과가 어떤 과정을 통해 다시 사용자에게 전달되는지를 알 수 있습니다.

이제 이러한 구조를 바탕으로, 실제로 웹페이지를 만들고 사용자의 입력을 처리하는 과정을 직접 구현해 보겠습니다.

다음 절에서는 VS Code를 이용하여 웹페이지를 구성하고 실행하는 방법을 단계별로 학습합니다.

© PMI. All rights reserved. 무단 복제 및 배포를 금합니다.

웹페이지 제작 및 실행하기

- **VS Code 설치 및 초기 설정하기**

지금까지 우리는 Google Colab과 같은 환경을 사용하여 Python 코드 작성과 실행을 진행해 왔습니다. Colab은 별도의 설치 없이 사용할 수 있어 매우 편리하지만, 웹페이지를 제작하는 과정에서는 HTML, CSS, JavaScript 파일을 직접 관리하고 여러 파일을 함께 구성해야 하기 때문에 보다 전문적인 개발 환경이 필요합니다

이러한 이유로 이번 장에서는 VS Code(Visual Studio Code)를 사용하여 웹페이지를 제작하고 실행하는 방법을 학습합니다. VS Code는 다양한 파일을 한 번에 관리할 수 있고, 웹 개발에 필요한 기능을 확장 프로그램을 통해 쉽게 추가할 수 있어 웹 서비스 개발에 매우 적합한 도구입니다.

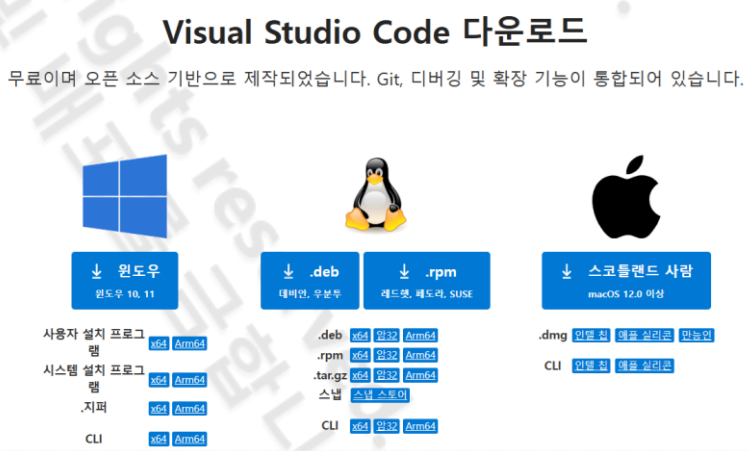
이번 절에서는 VS Code를 설치하고, 웹페이지를 보다 편리하게 개발할 수 있도록 기본 환경을 설정합니다.

1) VS Code 설치하기

VS Code는 무료로 제공되는 코드 편집기로, HTML, CSS, JavaScript와 같은 웹 개발 언어를 효율적으로 작성할 수 있는 도구입니다.

다음과 같은 순서로 설치를 진행합니다.

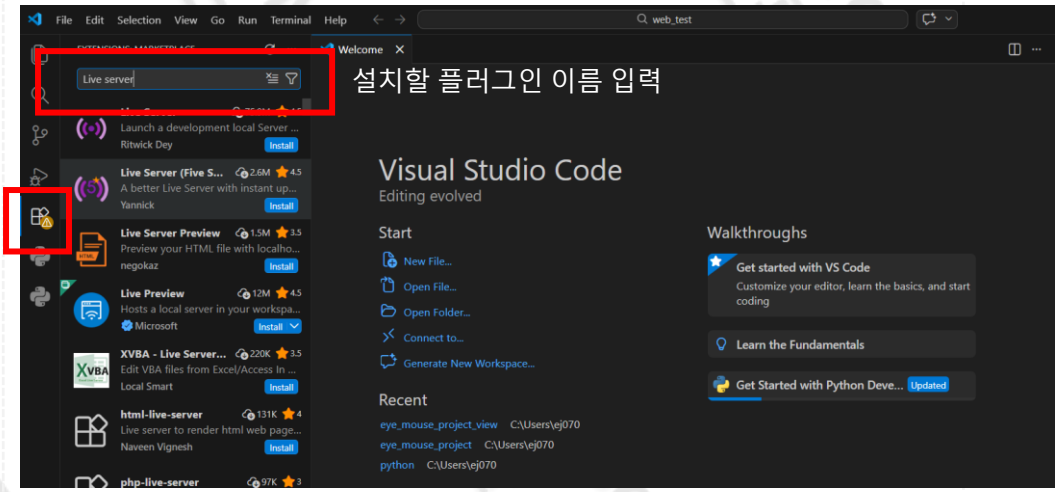
1. VS Code 공식 사이트(https://code.visualstudio.com/)에 접속합니다.
2. 자신의 운영체제에 맞는 설치 파일을 다운로드합니다.(Windows, macOS, Linux 지원)



2) 추천 플러그인 설치하기

VS Code를 설치한 후에는 웹 개발을 더욱 편리하게 하기 위해확장 기능(플러그인)을 추가로 설치합니다.

VS Code 실행 후, 좌측 메뉴의 확장 기능(Extensions) 아이콘을 클릭하여 다음 플러그인을 설치합니다.



설치할 플러그인 이름	설명
Live Server	HTML 파일을 브라우저에서 실시간으로 실행 및 미리보기
Prettier	코드 자동 정렬 (선택)
HTML CSS Support	HTML, CSS 문법 강조
JavaScript Language Basics	JavaScript 문법 지원 (기본 제공)

- 플러그인을 사용하는 이유

이러한 플러그인을 설치하면 다음과 같은 장점이 있습니다

- Live Server를 통해 웹페이지를 서버 형태로 실행할 수 있습니다.
- 코드 작성 시 자동 정렬과 문법 강조를 통해 가독성이 향상됩니다.

- 오류를 줄이고 개발 속도를 높일 수 있습니다.

특히 Live Server는 웹페이지를 수정할 때마다 자동으로 새로 고침 되므로, 변경 사항을 즉시 확인할 수 있어 매우 중요한 도구입니다.

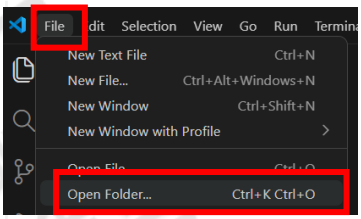
3) 새 프로젝트 폴더 만들기

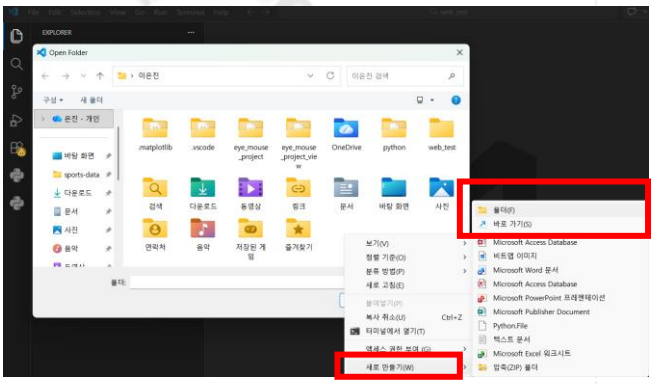
웹페이지를 제작할 때는 관련된 파일들을 하나의 폴더 안에서 관리하는 것이 중요합니다. 이를 통해 파일 간 연결을 쉽게 하고, 프로젝트 구조를 체계적으로 유지할 수 있습니다.

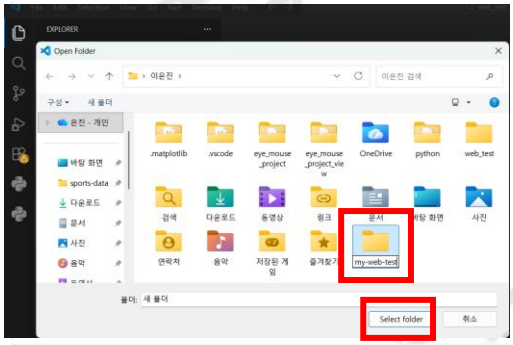
다음과 같은 순서로 새 프로젝트 폴더를 생성합니다.

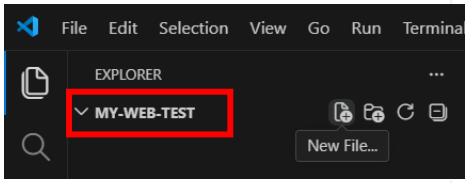
1. 원하는 위치에 새 폴더를 생성합니다. (예: my-web-test).
2. VS Code에서 상단 메뉴의 파일 → 폴더 열기를 선택한 후, 생성한 my-web-test 폴더를 열어줍니다.

이 과정을 통해 VS Code에서 해당 폴더를 하나의 프로젝트로 인식하게 됩니다.





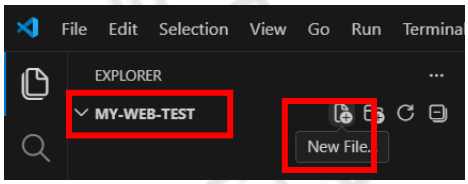


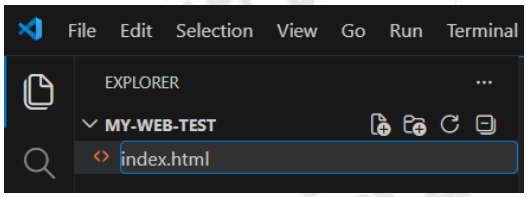


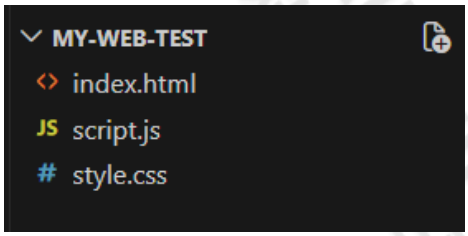
4) 기본 웹 파일 만들기

웹페이지는 하나의 파일로 구성되는 것이 아니라, 각 역할을 담당하는 여러 파일이 함께 동작하여 만들어집니다.

프로젝트 폴더 안에 다음과 같은 기본 파일 3개를 생성합니다.







- 각 파일의 역할

  - index.html : 웹페이지의 구조를 담당하는 파일로, 화면에 표시되는 내용을 정의합니다.
  - style.css : 웹페이지의 디자인을 담당하며, 색상, 글꼴, 여백 등을 설정합니다.
  - script.js : 웹페이지의 동작을 담당하며, 버튼 클릭이나 사용자 입력과 같은 기능을 처리합니다.

- 파일을 나누어 작업하는 이유

웹 개발에서는 기능을 분리하여 관리하는 것이 매우 중요합니다.

- 구조(HTML), 디자인(CSS), 동작(JavaScript)을 나누면 코드가 정리됩니다.
- 유지보수와 수정이 쉬워집니다.
- 실제 웹 서비스 개발에서도 동일한 구조를 사용합니다.

© PMI. All rights reserved. 무단 복제 및 배포를 금지합니다.

5) 웹페이지 코드 작성하기

앞에서 생성한 index.html, style.css, script.js 파일에 실제 코드를 입력하여 웹페이지를 완성합니다.

index.html 코드 작성

1. 작업 방법

- VS Code 왼쪽 탐색기에서 index.html 클릭
- 기존 내용이 있다면 모두 삭제
- 아래 코드를 작성 또는 Github 복사

- 코드 작성

```html
<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8">
  <title>첫 웹페이지</title>

  <!-- CSS 연결 -->
  <link rel="stylesheet" href="style.css">
</head>

<body>
  <h1>안녕하세요</h1>
  <p>ChatGPT가 알려준 웹 예제입니다.</p>

  <!-- 버튼 클릭 시 JS 함수 실행 -->
  <button onclick="sayHello()">눌러보세요</button>

  <!-- JS 연결 -->
  <script src="script.js"></script>
</body>
</html>
```

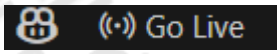
© PMI. All rights reserved. 무단 복제 및 배포를 금합니다.

웹페이지 제작 및 실행하기

- 코드 설명

- <!DOCTYPE html> : 현재 문서가 HTML5 형식임을 선언합니다.
- <html lang="ko"> : HTML 문서의 시작이며, 언어를 한국어로 설정합니다.
- <head> : 웹페이지의 설정 정보를 담는 영역입니다. (화면에는 보이지 않음).
- <meta charset="UTF-8"> : 한글이 깨지지 않도록 문자 인코딩을 설정합니다.
- <title>첫 웹페이지</title> : 브라우저 탭에 표시되는 제목입니다.
- <link rel="stylesheet" href="style.css"> : CSS 파일을 연결하여 디자인을 적용합니다.
- <body> : 사용자에게 실제로 보이는 내용을 작성하는 영역입니다.
- <h1>안녕하세요</h1> : 가장 큰 제목을 표시합니다.
- ... : 일반 문장을 출력하는 태그입니다.
- <button onclick="sayHello( )"> : 버튼 클릭 시 sayHello() 함수가 실행됩니다.
- <script src="script.js"></script> : JavaScript 파일을 연결하여 동작을 구현합니다.

웹페이지 제작 및 실행하기

- 코드 실행 결과
  - Vs code 오른쪽 하단에 Go Live 클릭 

style.css 코드 작성

1. 작업 방법

- VS Code 왼쪽 탐색기에서 style.css 클릭
- 기존 내용이 있다면 모두 삭제
- 아래 코드를 작성 또는 Github 복사

- 코드 작성

```
body {
  font-family: '맑은 고딕', sans-serif;
  background-color: #f5f5f5;
  padding: 30px;
}

h1 {
  color: #007acc;
}
```

© PMI. All rights reserved. 무단 복제 및 배포를 금합니다.

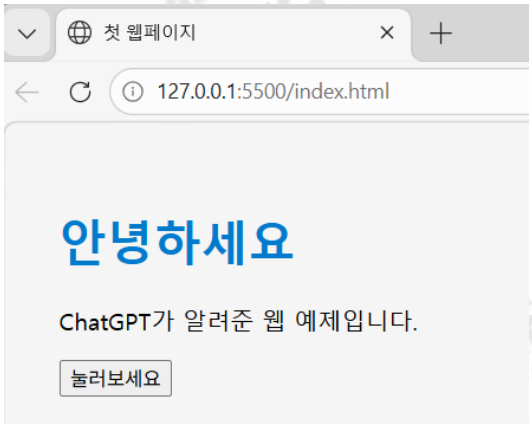
웹페이지 제작 및 실행하기

- 코드 설명

- body { ... } : 웹페이지 전체 영역에 스타일을 적용합니다.
- font-family: '맑은 고딕', sans-serif; : HTML 문서의 시작이며, 언어를 한국어로 설정합니다.
- background-color: #f5f5f5; : 배경색을 밝은 회색으로 설정합니다.
- padding: 30px; : 화면 가장자리와 내용 사이에 여백을 줍니다.
- h1 { color: #007acc; } : 제목(h1)의 글자 색을 파란색으로 변경합니다.

- 코드 실행 결과

  - Vs code 오른쪽 하단에 Go Live 클릭



style.css 코드 작성

- 코드 작성

```
function sayHello() {
  alert("Hello! ChatGPT 웹 예제입니다.");
}
```

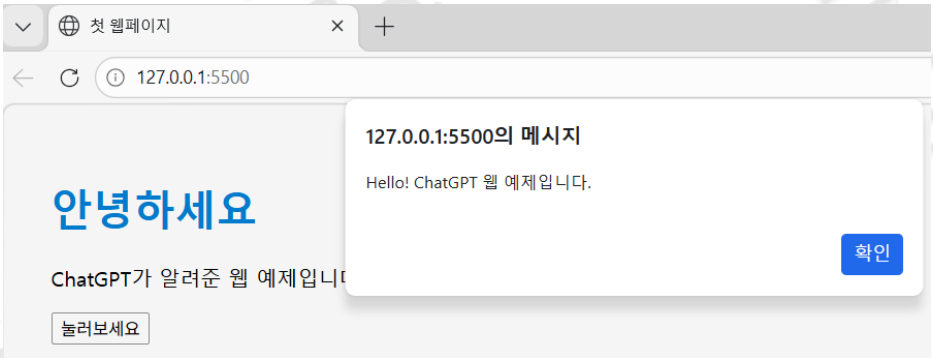
© PMI. All rights reserved. 무단 복제 및 배포를 금합니다.

웹페이지 제작 및 실행하기

- 코드 설명

- function sayHello() { ... } : sayHello라는 이름의 함수를 정의합니다. 이 함수는 버튼 클릭 시 실행됩니다.
- alert("Hello! ChatGPT 웹 예제입니다."); : 알림창(alert)을 띄워 메시지를 화면에 출력합니다.

- 코드 실행 결과
  - Vs code 오른쪽 하단에 Go Live 클릭
  - "눌러보세요"를 클릭



© PMI. All rights reserved. 무단 복제 및 배포를 금합니다.

Firebase 로그인/회원가입 구현

이번 단계에서는 단순히 하나의 웹페이지를 보여주는 것을 넘어서, 여러 페이지를 서로 연결하고 사용자 인증 기능을 추가하여 웹 서비스를 실제 서비스 형태에 가깝게 확장합니다. 이를 위해 첫 화면, 로그인 화면, 그리고 이동용 페이지를 각각 구성하고, 버튼을 클릭했을 때 원하는 페이지로 이동할 수 있도록 합니다. 또한 login.html 페이지에서는 Firebase를 활용하여 회원가입과 로그인 기능을 구현합니다.

앞에서 만든 웹페이지는 하나의 화면만 동작하는 가장 기본적인 예제였습니다. 하지만 실제 웹 서비스는 여러 개의 페이지가 서로 연결되어 있으며, 사용자의 선택에 따라 다른 화면으로 이동할 수 있어야 합니다. 또한 단순히 정보를 제공하는 것을 넘어, 회원가입과 로그인 기능을 통해 사용자를 구분하고 서비스 기능을 다르게 제공하는 경우가 많습니다.

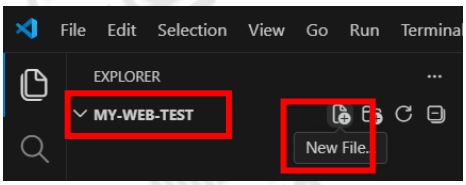
따라서 이번 단계는 정적인 웹페이지에서 더 나아가 페이지 이동과 사용자 인증 기능이 있는 웹 서비스 구조를 이해하는 중요한 과정입니다.

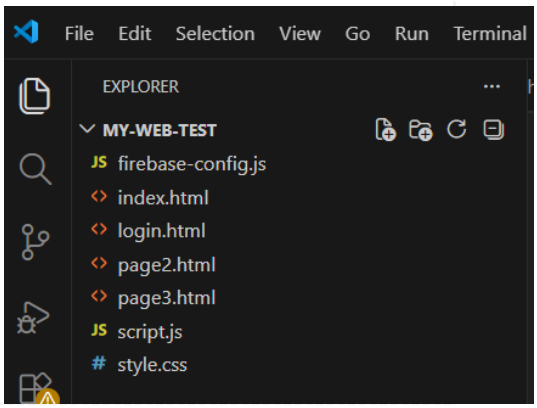
- 프로젝트 구조 이해하기

  여러 페이지와 사용자 인증 기능이 추가되면 웹 서비스는 하나의 파일이 아닌 여러 파일이 서로 연결된 구조로 동작하게 됩니다. 이때 각 파일은 서로 다른 역할을 수행하며, 이들이 유기적으로 연결되어 하나의 서비스로 동작하게 됩니다.

  따라서 이러한 기능을 올바르게 구현하기 위해서는 각 파일의 역할을 이해하고, 프로젝트를 체계적으로 구성하는 것이 중요합니다.

  다음과 같은 구조로 프로젝트를 구성합니다.





Firebase 로그인/회원가입 구현

프로젝트 구조 설명

- index.html : 웹사이트의 첫 화면(메인 페이지)
- login.html : 회원가입 및 로그인 기능을 수행하는 페이지
- page2.html, page3.html : 버튼 클릭 시 이동하는 페이지
- style.css : 전체 페이지 디자인
- script.js : 공통 JavaScript 기능
- firebase-config.js : Firebase와 웹페이지를 연결하는 설정 파일

index.html 코드 작성

1. 작업 방법

- VS Code 왼쪽 탐색기에서 index.html 클릭
- 기존 내용이 있다면 모두 삭제
- 아래 코드를 작성 또는 Github 복사

- 코드 작성

```html
<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8">
  <title>메인 페이지</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <h1>메인 페이지</h1>
  <button onclick="location.href='login.html'">
    회원가입 / 로그인
  </button>

  <button onclick="location.href='page2.html'">
    페이지 2
  </button>
```

© PMI. All rights reserved. 무단 복제 및 배포를 금합니다.

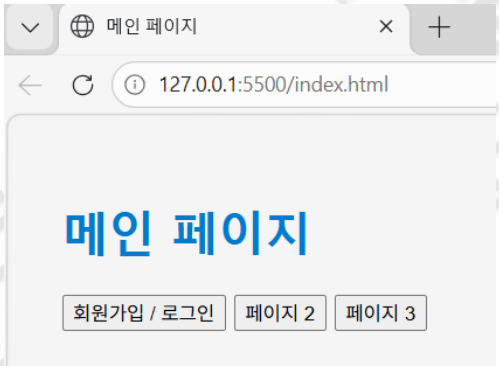
```
    <button onclick="location.href='page3.html'">
      페이지 3
    </button>

  </body>
  </html>
```

- 코드 설명

- <button onclick="location.href='login.html'"> : 버튼 클릭 시 login.html 페이지로 이동
- <button onclick="location.href='page2.html'"> : 버튼 클릭 시 page2.html로 이동
- <button onclick="location.href='page3.html'"> : 버튼 클릭 시 page3.html로 이동

- 코드 실행 결과
  - Vs code 오른쪽 하단에 Go Live 클릭



© PMI. All rights reserved. 무단 복제 및 배포를 금합니다.

Firebase 로그인/회원가입 구현

- Firebase 프로젝트 생성 및 설정하기

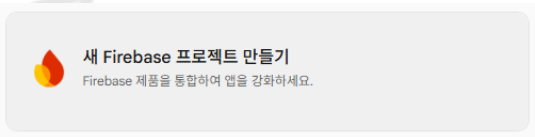
웹페이지에서 회원가입과 로그인 기능을 구현하기 위해서는 사용자의 정보를 저장하고 인증을 처리할 수 있는 시스템이 필요합니다.

하지만 이러한 기능을 직접 서버로 구현하는 것은 복잡하고 시간이 많이 소요되기 때문에, 이번 실습에서는 Firebase를 활용하여 간단하게 사용자 인증 기능을 구현합니다. Firebase는 별도의 서버 구축 없이도 회원가입과 로그인 기능을 제공하는 클라우드 기반 서비스로, 웹 애플리케이션에서 쉽게 사용할 수 있습니다. 이제 Firebase를 사용하기 위한 프로젝트를 생성하고,웹페이지와 연결하는 과정을 단계별로 진행해 보겠습니다.

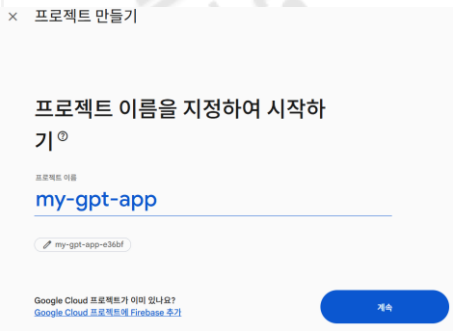
Firebase 프로젝트 만들기

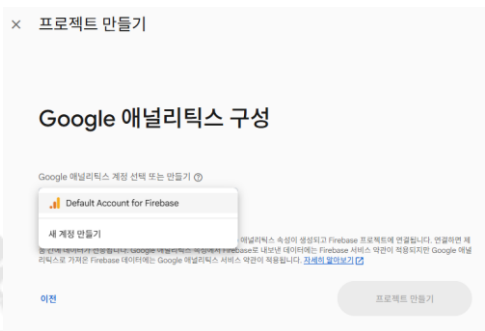
Firebase를 사용하기 위해서는 먼저 웹 애플리케이션을 관리할 수 있는 프로젝트를 생성해야 합니다. Firebase 프로젝트는 사용자 인증, 데이터 저장 등 웹 서비스에서 필요한 기능들을 관리하는 기본 단위입니다.

1. 작업 방법
   - Firebase 콘솔[https://console.firebase.google.com]에 접속합니다.
   - "프로젝트 만들기" 버튼을 클릭합니다.



   - 프로젝트 정보를 입력합니다.[프로젝트 이름 : 예) my-gpt-app]





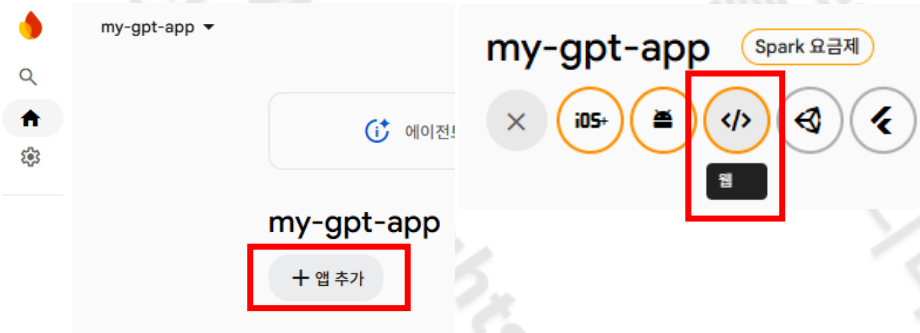
Firebase 로그인/회원가입 구현

웹 앱 등록하기

Firebase 프로젝트를 생성한 후에는 웹페이지에서 Firebase 기능을 사용할 수 있도록 웹 애플리케이션을 Firebase에 등록해야 합니다. 이 과정에서는 Firebase와 웹 페이지를 연결하기 위한 설정 정보를 발급받게 됩니다.

1. 작업 방법

- Firebase 프로젝트 생성 후, 상단 메뉴에서 웹 아이콘(</>)을 클릭합니다.



- 웹 앱 정보를 입력합니다.
  - 앱 이름 : 자유롭게 입력 (예: frontend-app)
  - Firebase Hosting 설정 : 체크하지 않아도 됨
- 등록을 완료하면 Firebase SDK 설정 코드가 화면에 표시됩니다.

Firebase 로그인/회원가입 구현

2 Firebase SDK 추가

◉ npm 사용 ○ <script> 태그 사용

이미 npm 및 모듈 번들러(예: Webpack 또는 Rollup)를 사용 중인 경우 다음 명령어를 실행하면 최신 SDK를 설치할 수 있습니다(자세히 알아보기).

```
$ npm install firebase
```

그런 다음 Firebase를 초기화하여 사용하려는 제품의 SDK를 사용하세요.

```
// Import the functions you need from the SDKs you need
import { initializeApp } from "firebase/app";
import { getAnalytics } from "firebase/analytics";
// TODO: Add SDKs for Firebase products that you want to use
// https://firebase.google.com/docs/web/setup#available-libraries

// Your web app's Firebase configuration
// For Firebase JS SDK v7.20.0 and later, measurementId is optional
const firebaseConfig = {
  apiKey: "AIzaSyBq-8A[illegible]8o",
  authDomain: "my-gpt-[illegible]",
  projectId: "my-gpt-a[illegible]",
  storageBucket: "my-g[illegible]ge.app",
  messagingSenderId: "[illegible]",
  appId: "1:4208020316[illegible]f8b",
  measurementId: "G-ZD[illegible]"
};

// Initialize Firebase
const app = initializeApp(firebaseConfig);
const analytics = getAnalytics(app);
```

- apiKey: Firebase 서비스를 사용하기 위한 인증 키
- authDomain : 로그인(인증)에 사용되는 도메인
- projectId : Firebase 프로젝트의 고유 이름
- storageBucket : 알림(메시지) 기능에 사용되는 ID
- appId : 웹 애플리케이션의 고유 식별 ID
- measurementId : 사용자 활동 분석(Analytics)을 위한 ID

firebase-config.js 파일 작성하기

웹 앱 등록 후 표시되는 Firebase 설정 정보를 이용하여 웹페이지와 Firebase를 연결하는 설정 파일을 작성합니다.

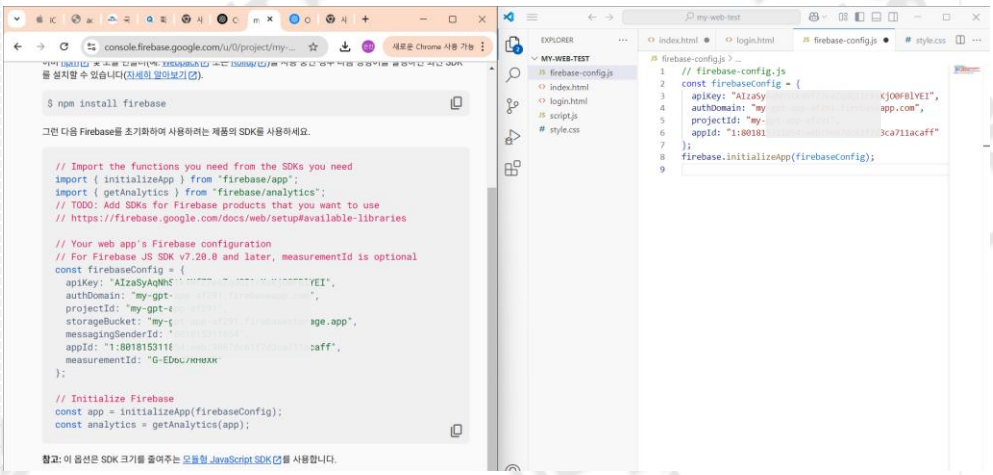
1. 작업 방법
   - Firebase에서 제공된 설정 코드(firebaseConfig)를 복사합니다.

Firebase 로그인/회원가입 구현

- VS Code에서 firebase-config.js 파일에 복사한 코드 추가한다.

```
const firebaseConfig = {
  apiKey: "복사한 키",
  authDomain: "xxx.firebaseapp.com",
  projectId: "xxx",
  appId: "xxx"
};

firebase.initializeApp(firebaseConfig);
```

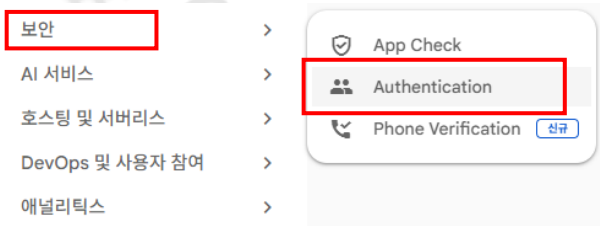


Firebase 인증(Authentication) 활성화하기

Firebase에서 회원가입과 로그인 기능을 사용하기 위해서는 인증(Authentication) 기능을 활성화해야 합니다. 이 과정에서는 이메일과 비밀번호를 이용한 로그인 방식을 설정합니다.

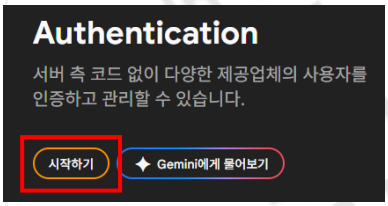
1. 작업 방법

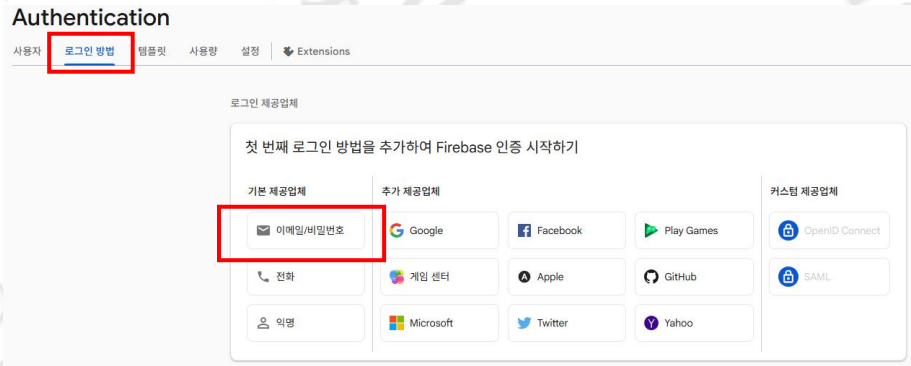
- Firebase 콘솔 왼쪽 메뉴에서 보안 - Authentication을 클릭합니다.

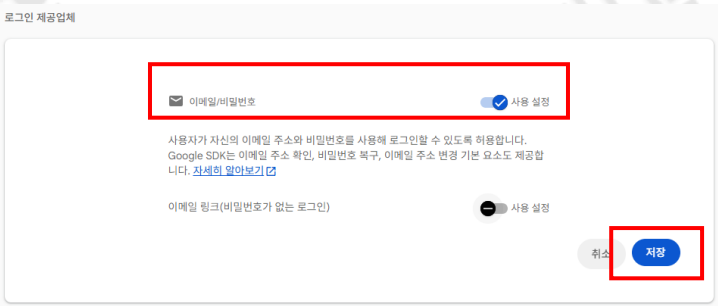


Firebase 로그인/회원가입 구현

- "시작하기" 버튼을 클릭합니다.
- 상단 메뉴에서 "로그인 방법" 탭으로 이동합니다.
- "이메일/비밀번호" 항목을 선택합니다.
- 사용 설정을 ON으로 변경한 후 저장합니다.







- Authentication : 사용자 회원가입 및 로그인 기능을 제공하는 Firebase 서비스
- 이메일/비밀번호 로그인 방식 : 가장 기본적인 사용자 인증 방법

© PNU. All rights reserved. 무단 복제 및 배포를 금합니다.

Firebase 로그인/회원가입 구현

- 로그인 및 회원가입 기능 구현하기

이번 단계에서는 Firebase를 활용하여 웹페이지에서 회원가입과 로그인 기능을 직접 구현합니다. 앞에서 Firebase 설정과 인증 기능을 활성화했기 때문에, 이제 login.html에서 사용자 인증 기능을 실제로 동작 시킬 수 있습니다.

login.html 파일 작성하기

login.html은 사용자가 회원가입과 로그인을 수행하는 페이지입니다. 이 페이지에는 이메일과 비밀번호를 입력하는 입력 창, 회원가입 버튼, 로그인 버튼이 포함됩니다. 또한 Firebase에서 제공하는 인증 기능을 사용하기 위해 Firebase 관련 스크립트와 설정 파일도 함께 연결해야 합니다. 즉, 이 페이지는 단순히 입력창만 있는 화면이 아니라 사용자 입력, Firebase 인증 처리, 결과 반환의 흐름이 실제로 이루어지는 핵심 페이지입니다.

1. 작업 방법
   - VS Code 왼쪽 탐색기에서 login.html 파일을 생성합니다.
   - 파일을 클릭한 뒤, 기존 내용이 있다면 모두 삭제합니다.
   - 아래 코드를 작성합니다.

- 코드 작성

```html
<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8">
  <title>로그인</title>

  <!-- Firebase SDK -->
  <script src="https://www.gstatic.com/firebasejs/9.22.2/
              firebase-app-compat.js"></script>
  <script src="https://www.gstatic.com/firebasejs/9.22.2/
              firebase-auth-compat.js"></script>

  <!-- Firebase 설정 -->
  <script src="firebase-config.js"></script>
</head>
```

© PMI. All rights reserved. 무단 복제 및 배포를 금합니다.

Firebase 로그인/회원가입 구현

```
<body>
  <h2>회원가입</h2>
  <input id="signup-email" placeholder="이메일">
  <input id="signup-password" type="password"
placeholder="비밀번호">
  <button onclick="signup()">회원가입</button>

  <h2>로그인</h2>
  <input id="login-email" placeholder="이메일">
  <input id="login-password" type="password“
         placeholder="비밀번호">
  <button onclick="login()">로그인</button>

  <script>
    const auth = firebase.auth();

    function signup() {
      const email = document.getElementById('signup-email’).
                    value;
      const password = document.getElementById('signup
                       -password’).value;

      auth.createUserWithEmailAndPassword(email, password)
        .then(() => alert("회원가입 완료!"))
        .catch(error => alert(error.message));
    }

    function login() {
      const email = document.getElementById('login
                    -email').value;
      const password = document.getElementById('login
                       -password').value;

      auth.signInWithEmailAndPassword(email, password)
        .then(() => {
          alert("로그인 성공!");
          location.href = "page2.html"; // 로그인 후 이동
        })
        .catch(error => alert(error.message));
    }
  </script>

</body>
</html>
```

Firebase 로그인/회원가입 구현

- 코드 설명

1) Firebase 연결 관련

- firebase-app-compat.js : Firebase 기본 기능 사용
- firebase-auth-compat.js : 회원가입 및 로그인 기능 사용
- firebase-config.js : Firebase 프로젝트와 웹페이지 연결 설정

2) HTML 기본 구조

- <!DOCTYPE html> : HTML5 문서 형식 선언
- <html lang="ko"> : 한국어 웹페이지 설정
- <meta charset="UTF-8"> : 한글 깨짐 방지
- <title>로그인</title> : 브라우저 탭에 표시되는 제목

3) 회원가입 영역

- <h2>회원가입</h2> : 회원가입 제목 표시
- <input id="signup-email"> : 이메일 입력창
- <input id="signup-password" type="password"> : 비밀번호 입력창 (숨김 처리)
- <button onclick="signup()">회원가입</button> : 클릭 시 회원가입 함수 실행

4) 로그인 영역

- <h2>로그인</h2> : 로그인 제목 표시
- <input id="login-email"> : 로그인 이메일 입력창
- <input id="login-password" type="password"> : 로그인 비밀번호 입력창
- <button onclick="login()">로그인</button> : 클릭 시 로그인 함수 실행

5) JavaScript (기능 구현)

① Firebase 인증 객체

- const auth = firebase.auth(); : Firebase 인증 기능 사용 준비

② 회원가입 함수

- function signup() : 회원가입 기능 정의

Firebase 로그인/회원가입 구현

- document.getElementById('signup-email').value : 입력한 이메일 가져오기
- document.getElementById('signup-password').value : 입력한 비밀번호 가져오기
- auth.createUserWithEmailAndPassword(email, password) : Firebase에 계정 생성
- .then(() => alert("회원가입 완료!")) : 성공 시 알림 표시
- .catch(error => alert(error.message)) : 실패 시 오류 메시지 표시

③ 로그인 함수

- function login() : 로그인 기능 정의
- auth.signInWithEmailAndPassword(email, password) : Firebase로 로그인 시도
- .then(() => { ... }) : 로그인 성공 시 실행
- alert("로그인 성공!") : 로그인 성공 메시지 출력
- location.href = "page2.html" : 페이지 이동
- .catch(error => alert(error.message)) : 로그인 실패 시 오류 메시지 표시

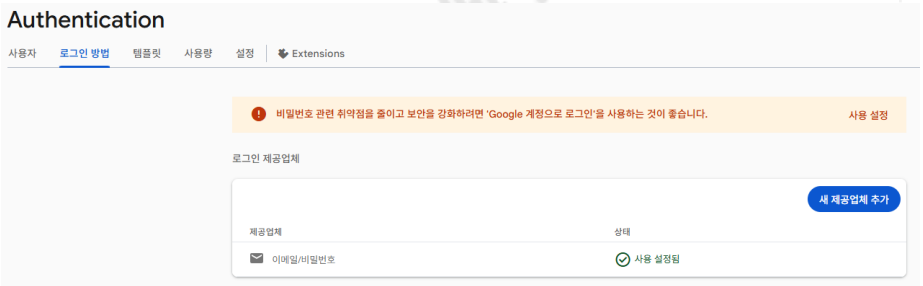
- 코드 실행 결과
  - Vs code 오른쪽 하단에 Go Live 클릭

  - Firebase 인증 설정 확인

Firebase 콘솔의 Authentication - 로그인 방법 메뉴에서 "이메일/비밀번호" 방식이 사용 설정된 상태인지 확인합니다. 이 설정이 활성화되어 있어야 웹 페이지에서 회원가입과 로그인 기능을 사용할 수 있습니다

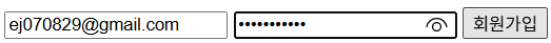
© PMI. All rights reserved. 무단 배포 및 재판매 금지

Firebase 로그인/회원가입 구현

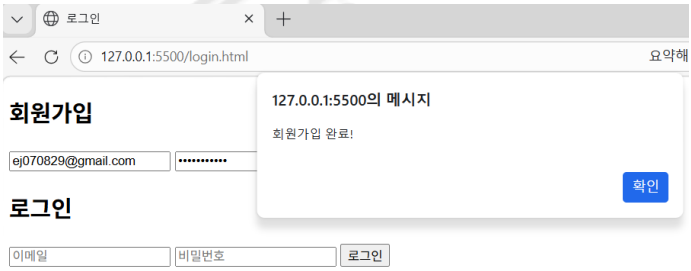


- 웹페이지에서 회원가입 실행

웹페이지에서 이메일과 비밀번호를 입력하고 회원가입 버튼을 클릭합니다.



회원가입이 정상적으로 처리되면 다음과 같은 알림창이 표시됩니다.



- Firebase 사용자 생성 확인

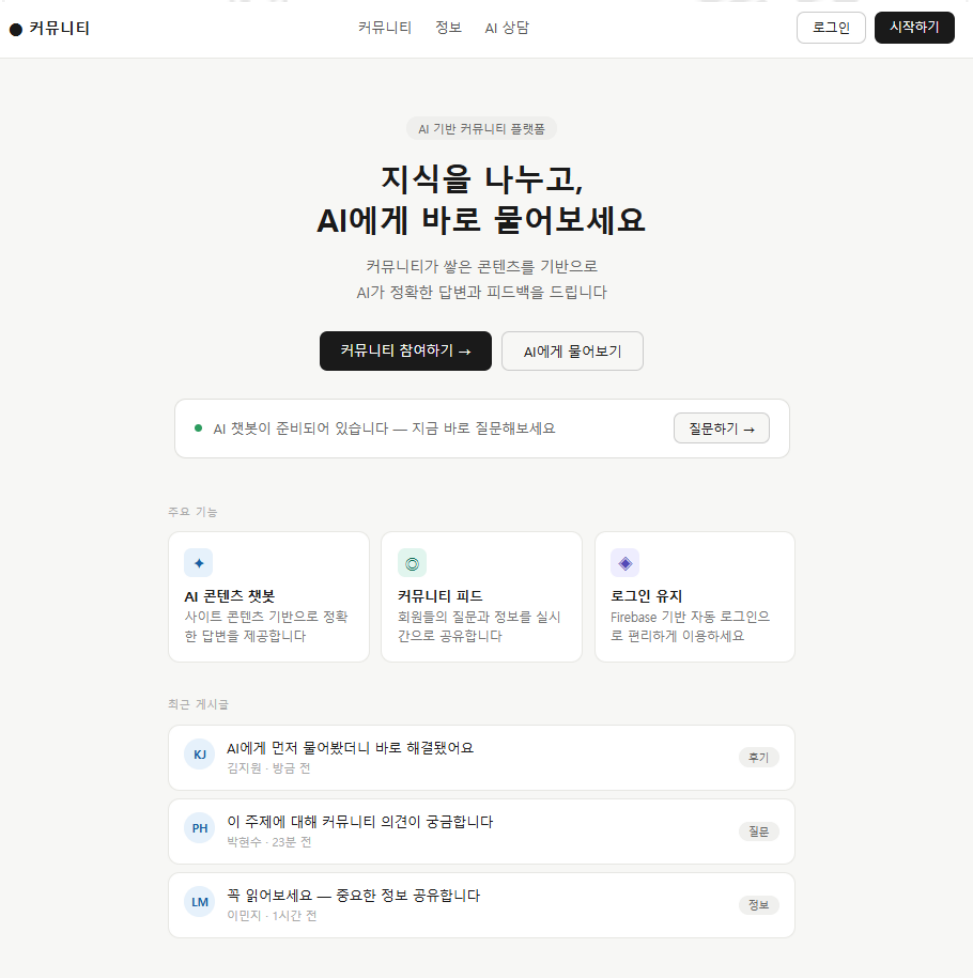
회원가입이 완료된 후, 다시 Firebase 콘솔로 이동하여 Authentication - 사용자 탭을 확인합니다. 입력한 이메일이 사용자 목록에 추가되어 있다면 회원가입이 정상적으로 완료된 것입니다.

Firebase 로그인/회원가입 구현

[AI를 활용한 실습]

1. Index.html 화면 디자인하기

AI를 활용해서 아래와 같이 화면을 디자인해봅니다.



© PNU. All rights reserved. 무단 복제 및 배포 금지

로그인 상태 유지 및 로그 아웃 기능 구현

앞 단계에서는 Firebase Authentication을 활용하여 회원가입과 로그인 기능을 구현하였다. 이를 통해 사용자가 이메일과 비밀번호를 이용해 웹페이지에 로그인할 수 있게 되었다. 하지만 실제 웹서비스에서는 단순히 로그인만 되는 것으로는 충분하지 않다. 사용자가 한 번 로그인한 뒤 페이지를 새로 고치거나 다시 접속했을 때도 로그인 상태가 유지되어야 하며, 필요할 경우 직접 로그아웃할 수 있는 기능도 함께 제공되어야 한다.

이번 단계에서는 Firebase에서 제공하는 인증 상태 확인 기능을 활용하여, 현재 사용자가 로그인된 상태인지 확인하고 그 결과에 따라 화면을 다르게 동작하도록 구현한다. 또한 로그아웃 기능을 추가하여 사용자가 서비스를 안전하게 종료할 수 있도록 구성한다.

즉, 앞에서 구현한 회원가입과 로그인 기능이 사용자 인증의 시작 단계였다면, 이번 단계는 로그인 이후의 상태를 관리하는 기능을 완성하는 과정이라고 볼 수 있다. 이 기능을 적용하면 웹페이지는 단순한 로그인 실습 화면을 넘어, 실제 서비스와 유사한 사용자 인증 흐름을 갖추게 된다.

- 로그인 상태 유지가 필요한 이유

  사용자가 페이지를 새로고침해도 다시 로그인할 필요가 없도록 하기 위함.

  현재 로그인된 사용자인지 확인하여 맞는 화면을 보여주기 위함.

  실제 웹서비스와 같은 인증 흐름을 구현하기 위함.

- Firebase에서 인증 상태 확인하기

  Firebase는 현재 로그인 상태를 확인할 수 있는 기능을 제공한다. 이를 위해 onAuthStateChanged() 함수를 사용하면 사용자의 로그인 여부를 실시간으로 확인할 수 있다. 이 함수를 활용하면 로그인 상태일 때는 메인 페이지로 이동시키고, 로그아웃 상태일 때는 로그인 화면을 보여주는 방식으로 웹페이지를 구성할 수 있다

- 로그아웃 기능 추가하기

  로그인 상태가 유지되는 기능을 구현했다면, 반대로 사용자가 원할 때 현재 인증 상태를 해제할 수 있는 로그아웃 기능도 필요하다.

로그인 상태 유지 및 로그 아웃 기능 구현

- 회원 전용 페이지 작성하기

앞 단계에서 로그인 기능을 구현하였다면, 로그인에 성공한 사용자를 위한 회원 전용 페이지가 필요하다. 이 페이지에서는 현재 로그인한 사용자의 정보를 확인하고, 로그아웃 기능을 제공한다

이번 단계에서는 회원 전용 페이지를 구성하기 위해 다음과 같은 파일을 새로 생성한다.

- page2.html : 회원 전용 화면 구성

- page2.js : 로그인 상태 확인 및 로그아웃 기능 구현

- page2.css : 페이지 스타일 적용

이와 같이 HTML, JavaScript, CSS 파일을 분리하여 구성하면 기능과 디자인을 효율적으로 관리할 수 있다

이제 page2.html 파일부터 작성해보자.

- 코드 작성

```
<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8">
  <title>회원 전용 페이지</title>

  <!-- 스타일 적용 -->
  <link rel="stylesheet" href="page2.css">
  <!-- Firebase SDK -->
  <script
src="https://www.gstatic.com/firebasejs/9.22.2/firebase-
app-compat.js"></script>
  <script
src="https://www.gstatic.com/firebasejs/9.22.2/firebase-
auth-compat.js"></script>
  <!-- Firebase 설정 -->
  <script src="firebase-config.js"></script>
</head>

<body>
```

로그인 상태 유지 및 로그 아웃 기능 구현

```
  <h1>회원 전용 페이지</h1>
  <!-- 사용자 정보 출력 -->
  <p id="welcome-msg">로딩 중...</p>
  <!-- 로그아웃 버튼 -->
  <button onclick="logout()">로그아웃</button>

  <!-- JS 파일 연결 -->
  <script src="page2.js"></script>
</body>
</html>
```

- 코드 설명

1) Firebase 연결 관련

- firebase-app-compat.js : Firebase 기본 기능 사용
- firebase-auth-compat.js : 로그인/로그아웃 인증 기능 사용
- firebase-config.js : Firebase 프로젝트와 웹페이지 연결 설정

2) HTML 기본 구조

- <title>회원 전용 페이지</title> : 브라우저 탭 제목 설정
- <h1> : 페이지 제목 표시
- <p id="welcome-msg"> : 로그인한 사용자 정보를 출력하는 영역
- "로딩 중..." : JavaScript 실행 전 기본 안내 문구

3) 사용자 정보 출력 영역

- id="welcome-msg" : JavaScript에서 해당 요소를 찾아 로그인한 사용자 이메일을 표시

4) 로그아웃 버튼

- <button onclick="logout()"> : 클릭 시 page2.js 파일에 정의된 logout() 함수 실행

5) script 위치 (중요)

① <script src="page2.js"></script>를 body 하단에 위치

- Firebase 초기화 이후 JavaScript가 실행되도록 하기 위함

© PMI. All rights reserved. 무단 복제 및 배포를 금합니다.

로그인 상태 유지 및 로그 아웃 기능 구현

- page2.js 파일 작성하기

회원 전용 페이지에서 로그인 상태를 확인하고, 사용자 정보를 출력하며, 로그아웃 기능을 구현하기 위해 page2.js 파일을 작성한다.

- 파일 생성

VS Code에서 page2.js 파일을 생성합니다.

- 코드 작성

```
const auth = firebase.auth();

// 로그인 상태 확인
auth.onAuthStateChanged(user => {
  if (user) {
    document.getElementById('welcome-msg').textContent =
      user.email + ' 님, 환영합니다!';
  } else {
    alert('로그인이 필요합니다.');
    location.href = 'login.html';
  }
});

// 로그아웃 기능
function logout() {
  auth.signOut().then(() => {
    alert('로그아웃 되었습니다.');
    location.href = 'index.html';
  });
}
```

- 코드 설명

1) Firebase 인증 객체 생성

- const auth = firebase.auth( ); : Firebase의 인증 기능을 사용하기 위한 객체 생성

2) 로그인 상태 확인

- auth.onAuthStateChanged(user => { }) : 현재 사용자의 로그인 상태를 확인하는 함수

로그인 상태 유지 및 로그 아웃 기능 구현

로그인된 상태일 경우

- user 값이 존재하면 로그인 상태
- user.email을 이용하여 사용자 이메일 확인 가능

```
document.getElementById('welcome-msg').textContent =
  user.email + ' 님, 환영합니다!';
```

로그인되지 않은 상태일 경우

- user 값이 존재하면 로그인 상태
- user.email을 이용하여 사용자 이메일 확인 가능

```
alert('로그인이 필요합니다.');
location.href = 'login.html';
```

3) 로그아웃 기능

```
auth.signOut()
```

- auth.signOut() : Firebase에서 현재 로그인된 사용자 로그아웃

로그아웃 성공 시

```
alert('로그아웃 되었습니다.');
location.href = 'index.html';
```

- page2.css 파일 작성하기

회원 전용 페이지의 가독성과 사용자 경험을 높이기 위해 CSS를 활용하여 화면 디자인을 적용한다. 이번 단계에서는 page2.css 파일을 생성하여 페이지의 전체 스타일을 구성한다.

- 파일 생성

VS Code에서 page2.css 파일을 생성합니다.

- 코드 작성

© PMI. All rights reserved. 무단 복제 및 배포를 금합니다.

로그인 상태 유지 및 로그 아웃 기능 구현

```
body {
  font-family: '맑은 고딕', sans-serif;
  background-color: #f0f8ff;
  padding: 40px;
}

h1 {
  color: #333;
}

.welcome {
  margin-top: 10px;
  font-weight: bold;
  font-size: 18px;
  color: #007acc;
}

button {
  margin-top: 20px;
  padding: 10px 20px;
  background-color: #007acc;
  color: white;
  border: none;
  border-radius: 6px;
  cursor: pointer;
  font-size: 15px;
}

button:hover {
  background-color: #005fa3;
}
```

- 코드 설명

1) body { } : 전체 페이지 스타일

- font-family : 전체 글꼴 설정
- '맑은 고딕' : 한글에 적합한 기본 폰트
- background-color : 페이지 배경 색상 지정
- padding : 화면 내부 여백 설정 (콘텐츠가 가장자리와 떨어지도록)

© PMI. All rights reserved. 무단 복제 및 배포를 금합니다.

로그인 상태 유지 및 로그 아웃 기능 구현

2) h1{ } : 제목 스타일

- color : 제목 글자 색상 설정
- #333 : 너무 밝지도 어둡지도 않은 기본 텍스트 색상

3) .welcome { } : 사용자 메시지 스타일

- .welcome : 클래스 선택자 (사용자 정보 출력 영역에 적용)
- margin-top : 위쪽 여백
- font-weight : 글자 굵기 설정
- font-size : 글자 크기 확대
- color : 강조 색상 (파란색 계열)

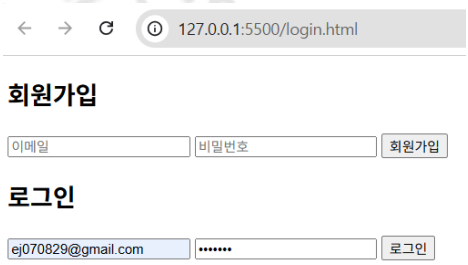
4) button { } : 버튼 스타일

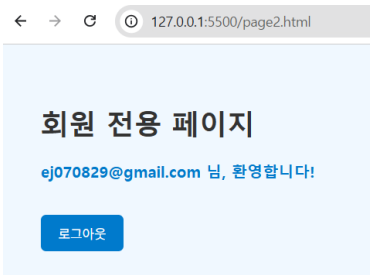
- padding : 버튼 내부 여백
- background-color : 버튼 배경 색상
- color : 버튼 글자 색상
- border-radius : 모서리를 둥글게 처리
- cursor: pointer : 마우스를 올렸을 때 클릭 가능한 형태로 변경

5) button:hover { } : 버튼 호버 효과

- :hover : 마우스를 올렸을 때 적용되는 스타일
- 버튼 색상을 어둡게 변경하여 사용자 인터랙션 효과 제공

- 코드 실행 결과





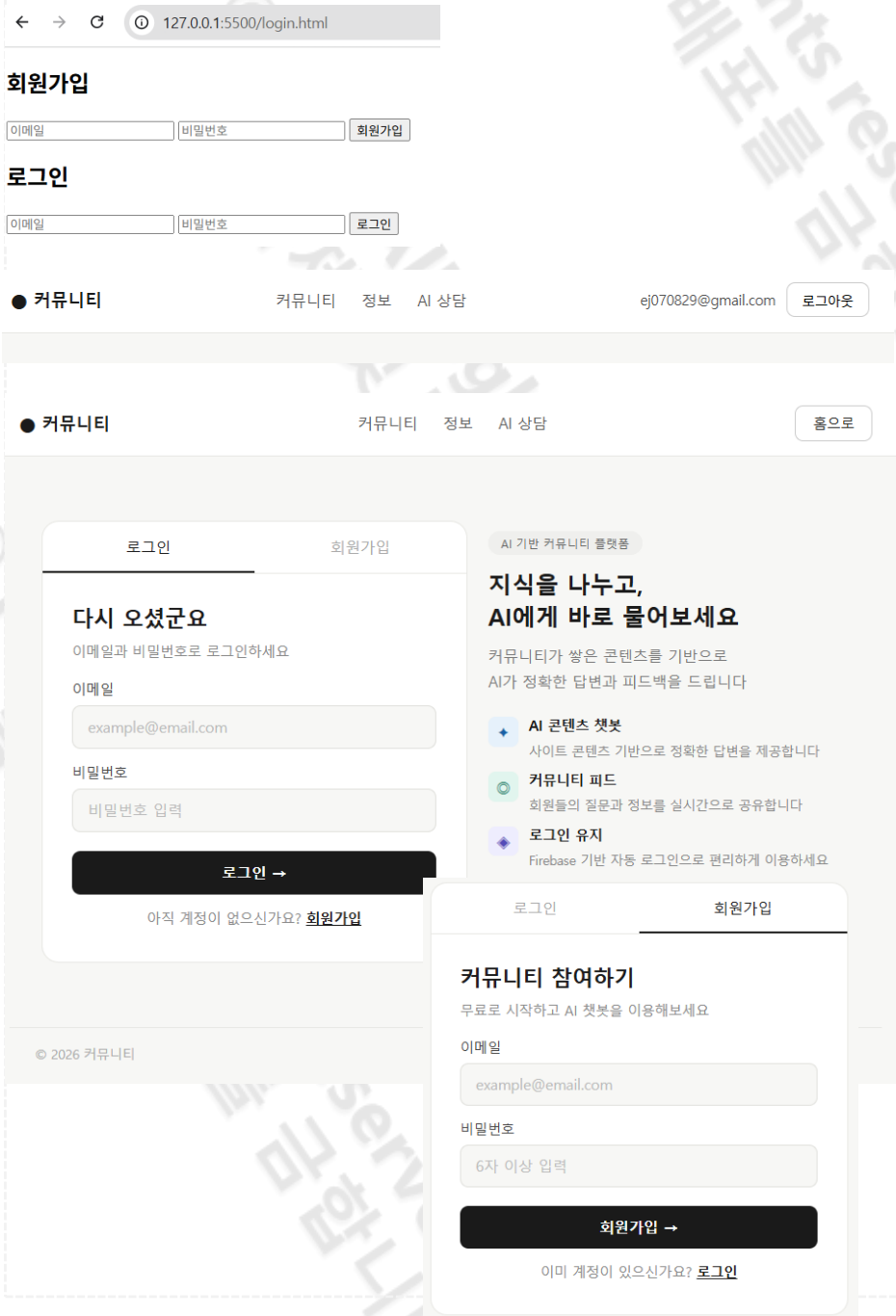
© PMI. All rights reserved. 무단 복제 및 배포를 금합니다.

로그인 상태 유지 및 로그 아웃 기능 구현

[AI를 활용한 실습]

1. login.html 화면 디자인하기

AI를 활용해서 아래와 같이 화면을 디자인해봅니다.



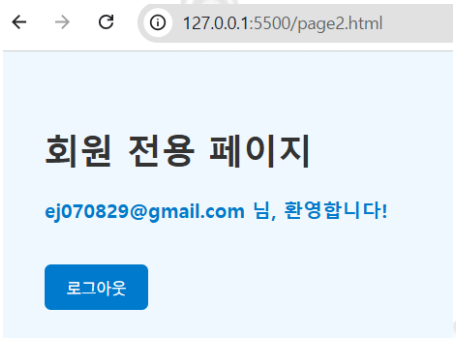
© PNU. All rights reserved. 무단 복제 및 배포를 금합니다.

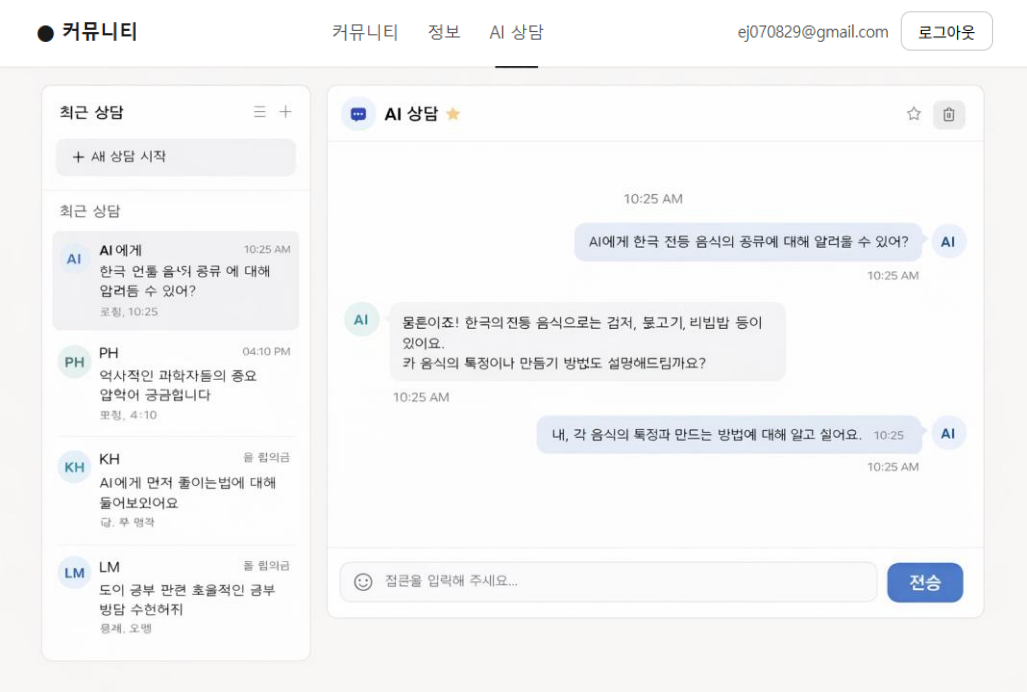
로그인 상태 유지 및 로그 아웃 기능 구현

[AI를 활용한 실습]

1. ai.html(page2.html) 화면 디자인하기

AI를 활용해서 아래와 같이 화면을 디자인해봅니다.





© PNU. All rights reserved. 무단 복제 및 배포를 금합니다.

GPT 연동 기능 구현

이번 단계에서는 기존에 구현한 AI 상담 화면에 실제 인공지능 기능을 연결하여, 사용자가 입력한 질문에 대해 AI가 응답을 생성하도록 구현한다. 앞 단계까지는 사용자 인터페이스(UI)와 로그인 기능을 중심으로 웹페이지의 구조를 구성하였다면, 이번 단계에서는 외부 API를 활용하여 웹페이지에 지능형 기능을 추가하는 과정을 학습한다.

웹페이지에서 AI 기능을 사용하기 위해서는 단순히 JavaScript만으로는 구현할 수 없으며, 외부 서버와의 통신이 필요하다. 이를 위해 Python 기반의 Flask 서버를 활용하여 사용자 요청을 처리하고, OpenAI API를 호출하여 응답을 받아오는 구조를 구성한다.

즉, 이번 단계는 프론트엔드(HTML, JavaScript)와 백엔드(Python)를 연결하여 하나의 완성된 AI 서비스 형태를 구현하는 과정이다.

- GPT 연동 기능 구조 이해하기

  GPT 기능이 포함된 웹서비스는 여러 파일이 유기적으로 연결되어 동작한다. 각 파일은 다음과 같은 역할을 수행한다.

  - HTML 파일 : 사용자 입력 및 결과 출력 화면 구성
  - JavaScript 파일 : 사용자 입력 처리 및 서버와 통신
  - Python 서버 : GPT API 호출 및 응답 반환
  - OpenAI API : 실제 인공지능 응답 생성

  이러한 구조를 통해 사용자의 입력이 서버를 거쳐 AI로 전달되고, 다시 결과가 웹페이지로 출력되는 흐름이 완성된다.

  다음과 같은 구조로 GPT 연동 기능을 구성한다.

© PMI. All rights reserved. 무단 복제 및 배포를 금합니다.

GPT 연동 기능 구현

- 구성도 설명

  1) ai.html : 사용자 입력과 AI 응답이 표시되는 화면을 구성하는 파일

  2) ai.js : 사용자 입력 처리 및 서버와 통신을 담당하는 JavaScript 파일

  3) server.py : Flask를 이용하여 GPT API와 연결하는 백엔드 서버

  4) requirements.txt : 프로젝트 실행에 필요한 Python 라이브러리 목록

  5) css/ai.css : AI 상담 화면의 디자인을 담당하는 스타일 파일

- GPT 연동의 동작 과정

앞에서 살펴본 GPT 연동 구조를 바탕으로, GPT 연동 기능이 실제로 어떻게 동작하는지 전체 흐름을 살펴보자. 각 파일은 서로 연결되어 하나의 과정으로 동작하며, 사용자의 입력이 AI의 응답으로 이어지는 흐름을 구성한다.

- GPT 연동 기능은 다음과 같은 순서로 진행된다.

  1) 사용자가 웹페이지에서 질문을 입력한다.

  2) JavaScript가 입력값을 서버로 전송한다.

  3) 서버는 해당 데이터를 OpenAI API로 전달한다.

  4) GPT가 질문에 대한 응답을 생성한다.

  5) 서버는 응답을 받아 다시 웹페이지로 전달한다.

  6) JavaScript가 응답을 화면에 출력한다.

- OpenAI API 키 발급하기

GPT 기능을 웹페이지에서 사용하기 위해서는 OpenAI에서 제공하는 API 키가 필요하다. API 키는 사용자의 요청을 OpenAI 서버에 전달할 때, 해당 요청이 정상적인 사용자로부터 발생했는지를 확인하는 인증 수단이다.

- API 키 발급 방법

  1) OpenAI 공식 사이트에 접속한다.(https://platform.openai.com)

  2) 로그인 후 API 키 관리 페이지로 이동한다.

  3) 새로운 API 키를 생성(Create new secret key)한다.

© PMI. All rights reserved. 무단 복제 및 배포를 금합니다.

GPT 연동 기능 구현

- API 키 사용 방법

발급받은 API 키는 서버 코드에서 다음과 같이 사용한다.

- Python 코드에 직접 입력하여 사용할 수 있다.
- 또는 .env 파일과 같은 환경 변수 파일에 저장하여 불러올 수 있다.

예를 들어, 서버 코드에서는 다음과 같은 형태로 API 키를 설정할 수 있다.

```
client = OpenAI(api_key="YOUR_API_KEY")
```

- API 키와 서버의 역할

웹페이지에서는 보안 문제로 인해 OpenAI API를 직접 호출하지 않고, 반드시 서버를 통해 요청을 전달해야 한다.

- 웹페이지(JavaScript)는 사용자 입력만 서버로 전달
- 서버(Python)는 API 키를 사용하여 GPT 호출
- 결과만 다시 웹페이지로 전달

예를 들어, 서버 코드에서는 다음과 같은 형태로 API 키를 설정할 수 있다.

- ai.html 파일 작성하기

```
<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width,
initial-scale=1.0">
  <title>GPT 연동 기능 구현</title>
  <link rel="stylesheet" href="css/ai.css">
</head>
<body>
  <div class="container">
    <h1>GPT 연동 기능 구현</h1>
    <p class="desc">질문을 입력하면 GPT가 답변을 생성합니다.
    </p>
```

© PNU. All rights reserved. 무단 복제 및 배포를 금합니다.

GPT 연동 기능 구현

```
    <textarea id="userInput" placeholder="질문을
입력하세요."> </textarea>

    <div class="button-wrap">
      <button onclick="sendToGPT()">GPT에게 보내기</button>
    </div>

    <h3>GPT 응답</h3>
    <div id="gptResponse" class="response-box">아직 응답이
없습니다.</div>
  </div>

  <script src="ai.js"></script>
</body>
</html>
```

- 코드 설명

1) 사용자 입력 영역 : <textarea id="userInput"

- 사용자가 질문을 입력하는 영역
- id="userInput"을 통해 JavaScript에서 입력값을 가져올 수 있다.

2) 버튼 클릭 이벤트 : <button onclick="sendToGPT( )">

- 버튼을 클릭하면 sendToGPT( ) 함수가 실행된다.
- 이 함수는 JavaScript에서 서버로 데이터를 보내는 역할을 한다.

3) 응답 출력 영역 :

- GPT의 응답 결과를 화면에 출력하는 영역
- JavaScript에서 해당 요소를 찾아 내용을 변경한다.

- ai.css 파일 작성하기

```
body {
  margin: 0;
  padding: 0;
  font-family: '맑은 고딕', sans-serif;
  background-color: #f4f6f8;
}
```

© PMI. All rights reserved. 무단 복제 및 배포를 금합니다.

GPT 연동 기능 구현

```
.container {
  width: 700px;
  margin: 50px auto;
  background-color: white;
  padding: 30px;
  border-radius: 12px;
  box-shadow: 0 4px 12px rgba(0, 0, 0, 0.08);
}

h1 {
  margin-top: 0;
  color: #222;
}

.desc {
  color: #666;
  margin-bottom: 20px;
}

textarea {
  width: 100%;
  height: 140px;
  padding: 12px;
  font-size: 16px;
  border: 1px solid #ccc;
  border-radius: 8px;
  resize: none;
  box-sizing: border-box;
}

.button-wrap {
  margin-top: 15px;
  margin-bottom: 25px;
}

button {
  background-color: #0078d4;
  color: white;
  border: none;
  padding: 12px 20px;
  font-size: 15px;
  border-radius: 8px;
  cursor: pointer;
}
```

© PMI. All rights reserved. 무단 복제 및 배포를 금합니다.

GPT 연동 기능 구현

```
button:hover {
  background-color: #005ea6;
}

.response-box {
  min-height: 120px;
  padding: 15px;
  background-color: #f9fafb;
  border: 1px solid #ddd;
  border-radius: 8px;
  line-height: 1.6;
  white-space: pre-wrap;
}
```

- ai.js 파일 작성하기

```
const chatHistory = [
  {
    role: "system",
    content: "너는 학생에게 친절하게 설명해주는 AI 선생님이야."
  }
];

async function sendToGPT() {
  const inputBox = document.getElementById("userInput");
  const responseBox = document.getElementById("gptResponse");
  const input = inputBox.value.trim();

  if (input === "") {
    alert("질문을 입력하세요.");
    return;
  }

  chatHistory.push({
    role: "user",
    content: input
  });

  responseBox.innerText = "응답을 생성하는 중입니다...";
```

© PMI. All rights reserved. 무단 복제 및 배포를 금합니다.

GPT 연동 기능 구현

```
    try {
      const response = await fetch("/chat", {
        method: "POST",
        headers: {
          "Content-Type": "application/json"
        },
        body: JSON.stringify({
          messages: chatHistory
        })
      });

      const data = await response.json();
      const reply = data.reply.content;

      chatHistory.push({
        role: "assistant",
        content: reply
      });

      responseBox.innerText = reply;
      inputBox.value = "";
    } catch (error) {
      responseBox.innerText = "오류가 발생했습니다: " +
error.message;
    }
  }
```

- 코드 설명

1) 대화 기록 저장 : const chatHistory = [

- chatHistory는 사용자와 AI의 대화 기록을 저장하는 배열이다.
- system 역할은 AI의 성격이나 말투를 설정한다.
- 이후 사용자 입력과 AI 응답이 계속 이 배열에 추가된다.

2) 입력값 가져오기

- inputBox : 사용자가 입력한 텍스트 영역
- responseBox : 결과를 출력할 영역
- trim( ) : 앞뒤 공백 제거

© PMI. All rights reserved. 무단 복제 및 배포를 금합니다.

GPT 연동 기능 구현

3) 사용자 질문 저장 : chatHistory.push({

- 사용자의 질문을 대화 기록에 추가
- 이후 GPT는 이 기록을 기반으로 응답을 생성

4) 로딩 메시지 출력 : responseBox.innerText

- GPT 응답이 오기 전 사용자에게 상태 안내

5) 서버로 요청 보내기

```
const response = await fetch("/chat", {
  method: "POST",
  headers: {
    "Content-Type": "application/json"
  },
  body: JSON.stringify({
    messages: chatHistory
  })
});
```

- fetch( ) → 서버와 통신하는 함수
- POST 방식 → 데이터를 서버로 전송
- JSON.stringify( ) : JavaScript 객체를 JSON 문자열로 변환

6) 서버 응답 처리

```
const data = await response.json();
const reply = data.reply.content;
```

- 서버에서 받은 JSON 데이터를 JavaScript 객체로 변환
- GPT가 생성한 답변(content)만 추출

7) AI 응답 저장

```
chatHistory.push({
  role: "assistant",
  content: reply
});
```

- AI의 응답도 대화 기록에 추가
- 다음 질문 시 문맥 유지 가능

© PMI. All rights reserved. 무단 복제 및 배포를 금합니다.

GPT 연동 기능 구현

- server.py 파일 작성하기

openai 패키지의 Python 사용 시, 키를 환경 변수로 읽어오도록 두는 편이 안전하다. OpenAI 도움말도 코드 하드코딩보다 OPENAI_API_KEY 환경 변수 사용을 권장한다.

```
from flask import Flask, request, jsonify,
send_from_directory
from openai import OpenAI
import os

app = Flask(__name__, static_url_path='')

client = OpenAI(api_key=os.environ.get("OPENAI_API_KEY"))

@app.route("/")
def index():
    return send_from_directory(".", "ai.html")

@app.route("/ai.js")
def serve_js():
    return send_from_directory(".", "ai.js")

@app.route("/css/<path:filename>")
def serve_css(filename):
    return send_from_directory("css", filename)

@app.route("/chat", methods=["POST"])
def chat():
    data = request.get_json()
    messages = data["messages"]

    try:
        response = client.chat.completions.create(
            model="gpt-4o-mini",
            messages=messages
        )

        reply = response.choices[0].message.content
```

© PMI. All rights reserved. 무단 복제 및 배포를 금합니다.

GPT 연동 기능 구현

```
        return jsonify({
            "reply": {
                "content": reply
            }
        })

    except Exception as e:
        return jsonify({
            "reply": {
                "content": f"에러 발생: {str(e)}"
            }
        })

if __name__ == "__main__":
    app.run(debug=True)
```

- requirements.txt 파일 작성하기

```
flask
openai
```

- 코드 실행 방법

OpenAI는 API 키를 브라우저가 아닌 서버 환경에 두고, 환경 변수 이름으로는 OPENAI_API_KEY를 권장한다. Windows에서는 setx OPENAI_API_KEY "<yourkey>" 방식으로 설정할 수 있다. 설정 후에는 새 터미널 창에서 실행해야 반영된다

```
pip install -r requirements.txt
python server.py
```

© PNU. All rights reserved. 무단 복제 및 배포를 금합니다.

© PMI. All rights reserved.
무단 복제 및 배포를 금합니다.